



Mahidol University
Wisdom of the Land

Programming Project

Presented by:

6388003 Phuriwat Angkoondittaphong

6388185 Bunradar Chatchaiyadech

Section 1

Submitted to:

Asst. Prof. Boonsit Yimwadsana

ITCS323 – Computer Data Communication

Semester 2 Academic year 2021

Faculty of Information Communication Technology

Mahidol University

CONTENTS

	Page
● Contents	1
● Parity bit	2
○ How to run the program	2
○ Screenshots of outputs from running your programs (testing results)	3
● CRC	7
○ How to run the program	7
○ Screenshots of outputs from running your programs (testing results)	7
● Checksum	11
○ How to run the program	11
○ Screenshots of outputs from running your programs (testing results)	11
● Hamming code	15
○ How to run the program	15
○ Screenshots of outputs from running your programs (testing results)	15

PARITY BIT

- How to run the program

To run the program,

```
javac Parity_bit.java
java Parity_bit
```

After the user runs the program, it will ask the user 4 questions.

- The first one is the array size.
- The second one is the maximum size of each dataword.
- Next is the datawords (the example of datawords form ["1010110"] for an dataword or for more than one dataword,
 1. The array has to be equal to the array size at the first question; otherwise, it will print "Please check your array size and the dataword again!" and the user will return to the beginning.
 2. Each element in the array should be more than or equal to 5, and less than or equal to the maximum size of each data word that the use has typed.; otherwise, it will print "Please check your word size again at index [a specify the array that the user should check]!" and the user will return to the beginning.
- The last question is the type of parity, of which there are 4 options for the user (press"1" one-dimensional-even, press"2" one-dimensional-odd, press"3" two-dimensional-even, press"4" two-dimensional-odd). If the user has typed others numbers, which are not in the list, it will print "Please choose the correct option!" and ask the user again and again.

After the user has typed the 4 questions, the program will print out the result.

- At the first line, it shows the datawords.
- At the second line, it shows the word size.
- At the third line, it shows the type of the parity.
- At the fourth line, it shows the array size.
- At the fifth line, it shows the codeword..

- At the last line, it shows that the codeword has been checked. It will show the pass message if there is no error, but it will show the fail message if there is some error.

(* If the user gets the result and wants to know the result of other datawords, the user needs to run the program again.)

- Screenshots of outputs from running your programs (testing results)
- For option 1 (one-dimensional-even)

```
--->Please input the array size
1
Please input the maximum size of each dataword
8
What is your datawords?
["11110000"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
1
Dataword: ["11110000"]
Word size: 8
Type of parity: one dimensional even
Array size: 1
Codeword: 11110000
Already check the parity codeword: Pass(1)
```

```
--->Please input the array size
1
Please input the maximum size of each dataword
9
What is your datawords?
["01010111"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
1
Dataword: ["01010111"]
Word size: 8
Type of parity: one dimensional even
Array size: 1
Codeword: 01010111
Already check the parity codeword: Pass(1)
```

- For option 2 (one-dimensional-odd)

```
--->Please input the array size
1
Please input the maximum size of each dataword
8
What is your datawords?
["00110011"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
2
Dataword: ["00110011"]
Word size: 8
Type of parity: one dimensional odd
Array size: 1
Codeword: 00110011
```

```

--->Please input the array size
1
Please input the maximum size of each dataword
9
What is your datawords?
["11100011"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
2
Dataword: ["11100011"]
Word size: 8
Type of parity: one dimensional odd
Array size: 1
Codeword: 111000110
Already check the parity codeword: Pass(1)

```

- For option 3 (two-dimensional-even)

```

--->Please input the array size
4
Please input the maximum size of each dataword
8
What is your datawords?
["1100111", "1011101", "0111001", "0101001"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
3
Dataword: ["1100111", "1011101", "0111001", "0101001"]
Word size: 28
Type of parity: two dimensional even
Array size: 4
Codeword: 11001111 10111011 01110010 01010011 01010101
Already check the parity codeword: Pass(1)

```

```

--->Please input the array size
3
Please input the maximum size of each dataword
8
What is your datawords?
["11111111", "01010101", "11111111"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
3
Dataword: ["11111111", "01010101", "11111111"]
Word size: 24
Type of parity: two dimensional even
Array size: 3
Codeword: 111111110 010101010 111111110 010101010
Already check the parity codeword: Pass(1)

```

```

--->Please input the array size
2
Please input the maximum size of each dataword
9
What is your datawords?
["011100", "10001000"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
3
Dataword: ["011100", "10001000"]
Word size: 14
Type of parity: two dimensional even
Array size: 2
Codeword: 000111001 100010000 100101001
Already check the parity codeword: Pass(1)

```

- For option 4 (two-dimensional-odd)

```

--->Please input the array size
4
Please input the maximum size of each dataword
8
What is your datawords?
["1100111", "1011101", "0111001", "0101001"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
4
Dataword: ["1100111", "1011101", "0111001", "0101001"]
Word size: 28
Type of parity: two dimensional odd
Array size: 4
Codeword: 11001110 10111010 01110011 01010010 10101011
Already check the parity codeword: Pass(1)

```

```

--->Please input the array size
3
Please input the maximum size of each dataword
8
What is your datawords?
["11111111", "01010101", "11111111"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
4
Dataword: ["11111111", "01010101", "11111111"]
Word size: 24
Type of parity: two dimensional odd
Array size: 3
Codeword: 11111111 01010101 11111111 101010101
Already check the parity codeword: Pass(1)

```

```
--->Please input the array size
2
Please input the maximum size of each dataword
9
What is your datawords?
["011100", "10001000"]
Type of parity: press"1" one-dimensional-even, press"2" one-dimensional-odd,
                press"3" two-dimensional-even, press"4" two-dimensional-odd
4
Dataword: ["011100", "10001000"]
Word size: 14
Type of parity: two dimensional odd
Array size: 2
Codeword: 000111000 100010001 011010110
Already check the parity codeword: Pass(1)
```

CRC

- How to run the program
 - Dependencies
 - Python 3.5+ (type hints)
 - For CRC generator use the following command:


```
python crc.py ${dataword} ${CRC type} [--word_size WORD_SIZE] [-v]
```
 - For CRC checker use the following command:


```
python crc.py ${codeword} ${CRC type} [--word_size WORD_SIZE] [-v]
```
 - `-v` is for verbosity. It will show the detail how to derive the output.
 - You can use the following command to find help in the terminal


```
python crc.py -h
```
 - Valid CRC type are


```
['CRC-32', 'CRC-24', 'CRC-16', 'Reversed CRC-16', 'CRC-8', 'CRC-4', 'test']
```

 - Test will use **1101** as a divisor like in the slide. [[ref](#) page 43]
 - You can use the following command to check on 10 datawords.(It should raise some exception of something wrong)


```
python test_crc.py
```
- Screenshots of outputs from running your programs (testing results)

101111000011000101101011100000010010001111001001100110111000100's CRC-8

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
101111000011000101101011100000010010001111001001100110111000100 CRC-8 --word_size 64
010111100001100010110101110000001001000111100100110011011100010000011111
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
010111100001100010110101110000001001000111100100110011011100010000011111 CRC-8 --word_size 64 --check
1
```

11100110100000001001001100111011110011001010000010011000101's CRC-4

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
11100110100000001001001100111011110011001010000010011000101 CRC-4 --word_size 64
000001110011010000000100100110111100110010100000100110001011001
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
000001110011010000000100100110111100110010100000100110001011001 CRC-4 --word_size 64 --check
1
```

100111111001100001111100110000100001011010001110010110111111's CRC-16

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100111111001100001111100110000100001011010001110010110111111 CRC-16 --word_size 64
0001001111100110000111110011000010000101101000111001011011111100000101010100
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
0001001111100110000111110011000010000101101000111001011011111100000101010100 CRC-16 --word_size 64 -
-check
1
```

1001000100011111101001110010010100000010100001000111011001110111's CRC-24

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
1001000100011111101001110010010100000010100001000111011001110111 CRC-24 --word_size 64
100100010001111110100111001001010000001010000100011101100111011110110101110000011100111
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100100010001111110100111001001010000001010000100011101100111011110110101110000011100111 CRC-24 --word_s
ize 64 --check
1
```

100000001010010001100001010011001101110110001111000111010001011's Reverse CRC-16

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100000001010010001100001010011001101110110001111000111010001011 "Reversed CRC-16" --word_size 64
01000000010100100011000010100110011011101100011110001110100010111101100110111111
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
01000000010100100011000010100110011011101100011110001110100010111101100110111111 "Reversed CRC-16" --wor
d_size 64 --check
1
```

101101011111001110001001100000110011110110111110010001011111110's CRC-32

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
101101011111001110001001100000110011110110111110010001011111110 CRC-32 --word_size 64
010110101111001110001001100000110011110110111110010001011111110010001100000111000100010001010
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
010110101111001110001001100000110011110110111110010001011111110010001100000111000100010001010 CRC-32
--word_size 64 --check
1
```

1110000100011010111000111000011111000001101110010011011011011011's CRC-4

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
1110000100011010111000111000011111000001101110010011011011011011 CRC-4 --word_size 64
11100001000110101110001110000111110000011011100100110110110111100
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
11100001000110101110001110000111110000011011100100110110110111100 CRC-4 --word_size 64 --check
1
```

100110110110011101011011011001110101001011111011100000001011100's CRC-8

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100110110110011101011011011001110101001011111011100000001011100 CRC-8 --word_size 64
010011011011001110101101100111010100101111101110000000101110011110001
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
010011011011001110101101100111010100101111101110000000101110011110001 CRC-8 --word_size 64 --check
1
```

1101100011001010001100011000100011010000001000100010001111101010's CRC-4

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
1101100011001010001100011000100011010000001000100010001111101010 CRC-4 --word_size 64
110110001100101000110001100010001101000000100010001000111110101000
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
110110001100101000110001100010001101000000100010001000111110101000 CRC-4 --word_size 64 --check
1
```

11101011000100100011011101010011100111110000101101111110110's CRC-16

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
11101011000100100011011101010011100111110000101101111110110 CRC-16 --word_size 64
000011101011000100100011011101010011100111110000101101111110110000011011111010
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
000011101011000100100011011101010011100111110000101101111110110000011011111010 CRC-16 --word_size 64 --check
1
```

CRC of 100100 using 1101 as divisor with verbosity [[ref](#) page 43 and 47]

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100100 test -v
Generate 100100's test
Codeword: modulo 2 division

-----
1101|100100000
 1101 XOR
-----
   1000
 1101 XOR
-----
   1010
 1101 XOR
-----
   1110
 1101 XOR
-----
   1100
 1101 XOR
-----
   001 <-- CRC or Syndrome

100100001
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\crc.py
100100001 test --check -v
Checking 100100001 with test
is codeword valid: modulo 2 division

-----
1101|100100001
 1101 XOR
-----
   1000
 1101 XOR
-----
   1010
 1101 XOR
-----
   1110
 1101 XOR
-----
   1101
 1101 XOR
-----
   000 <-- CRC or Syndrome
```

CHECKSUM

- How to run the program

- Dependencies

- Python 3.5+ (type hints)

- For CRC generator use the following command:

```
python checksum.py ${dataword} ${num block} [--word_size WORD_SIZE] [-v]
```

- For CRC checker use the following command:

```
python checksum.py ${codeword} ${num block} [--word_size WORD_SIZE] [-v]
```

- -v is for verbosity. It will show the detail how to derive the output.

- You can use the following command to find help in the terminal

```
python checksum.py -h
```

- You can use the following command to check on 10 datawords. (It should raise some exception of something wrong)

```
python test_checksum.py
```

- Screenshots of outputs from running your programs (testing results)

1011100010001100100000101010101011000100001100000010001001100101's checksum
8bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1011100010001100100000101010101011000100001100000010001001100101 8 --word_size 64
101110001000110010000010101010101100010000110000001000100110010100010001
```

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
101110001000110010000010101010101100010000110000001000100110010100010001 8 --word_size 64 --check
1
```

101110001011110010011110001011000000010000100110100111000111's checksum 4
bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
101110001011110010011110001011000000010000100110100111000111 4 --word_size 64
00001011100010111100100111100010110000000100001001101001110001110001
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
00001011100010111100100111100010110000000100001001101001110001110001 4 --word_size 64 --check
1
```

101101011111011110111001110000011010100110100010100100100101010's checksum

8bits/block with verbosity

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
101101011111011110111001110000011010100110100010100100100101010 8 --word_size 64 -v
    Adding Main bit
    00101010
    01001001
    11010001
    11010100
    11100000
    11011100
    11111011
    01011010
    -----
0000010100101001
    Adding Carry bit
    00101001
    00000101
    -----
    00101110 <-- Result from adding

    Complement it
    11010001 <-- Checksum/Syndrome
0101101011111011110111001110000011010100110100010100100100101011010001
```

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
0101101011111011110111001110000011010100110100010100100100101011010001 8 --word_size 64 --check -v
    Adding Main bit
    11010001
    00101010
    01001001
    11010001
    11010100
    11100000
    11011100
    11111011
    01011010
    -----
0000010111111010
    Adding Carry bit
    11111010
    00000101
    -----
    11111111 <-- Result from adding

    Complement it
    00000000 <-- Checksum/Syndrome
1
```

1000101111110000001110011011101011111101000111111100100100111100's checksum

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1000101111110000001110011011101011111101000111111100100100111100 16 --word_size 64
10001011111100000011100110111010111111010001111111001001001111000111111000
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1000101111110000001110011011101011111101000111111100100100111100011100111111000 16 --word_size 64 --check
1
```

1011110001110001111011001100101011011111101100110011011100100000's checksum
32bits/block.

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1011110001110001111011001100101011011111101100110011011100100000 32 --word_size 64
10111100011100011110110011001010110111111011001100110111001000001100011110110101101110000010100
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
10111100011100011110110011001010110111111011001100110111001000001100011110110101101110000010100 32 --word_size 64 --check
1
```

1101100010100011000100001101110010110000010001011101101110100001's checksum 10
bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1101100010100011000100001101110010110000010001011101101110100001 10 --word_size 64
11011000101000110001000011011100101100000100010111011011101000011100101001
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
11011000101000110001000011011100101100000100010111011011101000011100101001 10 --word_size 64 --check
1
```

1101000101000110111111000100101000000000000011000010001100100110's checksum
15bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
110100010100011011111100010010100000000000011000010001100100110 15 --word_size 64
011010001010001101111110001001010000000000011000010001100100110001111100001011
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
11010001010001101111110001001010000000000011000010001100100110001111100001011 15 --word_size 64 --check
1
```

1101101001110100010000000011001100100110101000010110100010001010's checksum
20bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1101101001110100010000000011001100100110101000010110100010001010 20 --word_size 64
110110100111010001000000001100110010011010100001011010001000101000001111000010111110
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
110110100111010001000000001100110010011010100001011010001000101000001111000010111110 20 --word_size 64 --check
1
```

1100110001100010111100011011011001101001101011110001101110000101's checksum

7bits/block

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1100110001100010111100011011011001101001101011110001101110000101 7 --word_size 64
11001100011000101111000110110110011010011010111100011011100001011100000
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
11001100011000101111000110110110011010011010111100011011100001011100000 7 --word_size 64 --check
1
```

1101001100001001001101010100100010011101101000110100111001000011's checksum

16bits/block with verbosity.

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
1101001100001001001101010100100010011101101000110100111001000011 16 --word_size 64 -v
Adding Main bit
  0100111001000011
  1001110110100011
  0011010101001000
  1101001100001001
-----
0000000000000011111010000110111
Adding Carry bit
  1111010000110111
  0000000000000001
-----
  1111010000111000 <-- Result from adding
Complement it
  0000101111000111 <-- Checksum/Syndrome
11010011000010010011010101001000100111011010001101001110010000110000101111000111
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> python .\checksum.py
11010011000010010011010101001000100111011010001101001110010000110000101111000111 16 --word_size 64 -v --check
Adding Main bit
  0000101111000111
  0100111001000011
  1001110110100011
  0011010101001000
  1101001100001001
-----
000000000000001111111111111110
Adding Carry bit
  1111111111111110
  0000000000000001
-----
  1111111111111111 <-- Result from adding
Complement it
  0000000000000000 <-- Checksum/Syndrome
1
```

HAMMING CODE

- How to run the program

To run the program,

```
javac Hamming_Code.java
java Hamming_Code
```

After the user runs the program, it will ask the user 2 questions.

- The first one is the dataword.
- The second one is the options that the user wants. Press "1" for generating the codeword or press "2" for checking the error position of the codeword

After the user has answered the questions,

1. If the user chooses option1, it will show the dataword, and the codeword.
2. If the user chooses option2, it will print the codeword and check the error position. If the error position is equal to 0, it means there is no error. If the error position is more than 0, it means there is an error position and it will show which position it is and print out the correct codeword.

- Screenshots of outputs from running your programs (testing results)

- For option 1 (generate the codeword)

110011's Hamming code

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
110011
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
1
Dataword: 110011
Code: 1100011101
```

1010111's Hamming code

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
1010111
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
1
Dataword: 1010111
Code: 10100110110
```

1000010's Hamming code

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
1000010
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
1
Dataword: 1000010
Code: 10010011010
```

- For option 2 (check the error position of the codeword)

Checking 1100011101 (110011's Hamming code)

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
1100011101
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
2
Codeword: 1100011101
There is no error
Data: 110011
```

Checking 1100001101 (110011's Hamming code with fault at position 5)

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
1100001101
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
2
Codeword: 1100001101
There is an error at position 5
The correct codeword should be:
110011
```

Checking 10100110110 (1010111's Hamming code)

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
10100110110
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
2
Codeword: 10100110110
There is no error
Data: 1010111
```

Checking 00100110110 (1010111's Hamming code with fault at position 11)

```
(base) PS I:\OneDrive - Mahidol University\ITCS323 Computer Data Communication\project> javac .\Hamming_Code.java && java Hamming_Code
What is your datawords?
00100110110
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,
2
Codeword: 00100110110
There is an error at position 11
The correct codeword should be:
1010111
```

Checking 10010011010 (1000010's Hamming code)

```
Choose the option: press "1" Generate the code word or press "2" check the error position of the codeword,  
2  
Codeword: 10010011010  
There is no error  
Data: 1000010
```